

Knowledge Graph for Emerging Technologies

Bachelor Practical Course: Data Engineering

Ngo Yan Abbie Fung✉ and Felipe Poviña Migliavacca✉

✉ abbie.fung@tum.de

✉ fpovina@correo.ugr.es

July 10, 2025

Abstract — This article presents the background, methodology, and evaluation of a modular pipeline for constructing a Knowledge Graph that maps emerging technologies to related entities, including start-ups, research papers, and skills. Drawing from diverse data sources, this pipeline standardizes, enriches, and links fragmented information across domains. The resulting Knowledge Graph enables explorations of connections between technologies, companies, skills, and publications, offering a valuable resource for trend analysis, innovation monitoring, and strategic decision making. This work highlights the decision making during the development process of building graph-based systems from real-world, semi-structured data.

1 Introduction

This project aims to build a **modular data engineering pipeline** to build a **knowledge graph**, that links emerging technologies to related entities such as **research papers**, **start-ups** and **required skills**. Keeping track of emerging technologies is often not an easy task, as what belongs in the category of emerging technologies changes from time to time. As a result, information is fragmented across data sources. This data engineering project takes data that spanned from 1994 to the year of 2025, from multiple databases, including arXiv, WikiData, Crunchbase, and Kaggle. The objective of this project is to build a modular, reusable, and scalable data pipeline.

2 Background

2.1 Knowledge Graph

Knowledge Graph is an organized representation of real-world entities and their relationships. [Stegeman, 2024] It consists of **nodes**, which denote and store details about entities, and **relationships**, that shows how the entities are related. In this project, we use nodes to represent a technology, a research paper, a skill, or a start-up; relationships are used when one data

entry mentions two different entities. For example, a research paper is written about the technology "3D printing", then the node representing said paper will be connected to the node representing "3D printing" with the relationship "mentions". We chose Knowledge Graph to store and represent data surrounding emerging technologies, as it allows high interconnectability among data, and provides convenience for cross-source entity linking, as we are utilizing multiple data sources in this project.

2.2 Entity Linking

Entity linking, also known as named entity disambiguation, is the task of assigning a unique identity to entities. [Ferré and Langlais, 2023] In this project, it involves matching possible names of technologies, skills, and start-ups. For example, for the technology "3D printing", it could be referred to as "additive manufacturing", "rapid prototyping", or "digital fabrication" in different data sources. Entity linking is the task of matching these names together so that when different entries mention these names, they are all represented by one single node.

2.3 Data Sources

In this project, we used multiple data sources. **Wikipedia** gives a curated list of emerging technologies, which is commercially available and community maintained. The list is completed by Organization for Economic Co-operation and Development (**OECD**)'s list of emerging technology. **WikiData** was used to obtain structured metadata for each technology. It enabled entity enrichment and disambiguation through its group-based API. **arXiv** is a curated research-sharing platform that hosts more than two million scholarly articles in eight subject areas, including physics, mathematics, computer science, quantitative biology, quantitative finance, statistics, electrical engineering and systems science, and economics. It provides an API interface that allows us to gain access to structured data on research papers.

Data on start-ups is obtained from **YCombinator** and **BrightData-scraped Crunchbase**. Y Combinator is a start-up accelerator and venture capital firm, and its dataset is especially valuable for its structured, high-quality records of early-staged companies; Crunchbase is a company that provides information on companies. Data on job skills comes from **Kaggle LinkedIn Job Skills dataset** that matches job postings to associated job skills and job titles.

We have also considered using **Crunchbase’s official API** for data records on start-ups, and **O*NET API** for job skills. However, even though the data provided by Crunchbase’s official API is very comprehensive and up-to-date, access to the full API requires a full subscription cost of 10,000 USD per year. This far exceeded our budget, therefore we turned to publicly available alternatives like YCombinator and BrightData to approximate the data. O*NET was evaluated as a potential source for job skills, however after careful inspection, we found out that the job skills it had classified under each occupation are too generic to gain meaningful insights. Instead, we turned to publicly available Kaggle dataset that matched job skills under job titles.

2.4 Neo4j and Cypher

Neo4j is a graph database management system. The data elements Neo4j stores are nodes, with edges connecting them. It was chosen for our project, as it has strong support for knowledge graphs, efficient querying and strong ecosystem. Its query language **Cypher** provides an intuitive and expressive syntax to explore the graph. Neo4j also has built in visualization tool, making manual exploration and analysis convenient.

3 Methods

3.1 Data Extraction

With the curated list of emerging technologies from Wikipedia and OCED, we used WikiData API to directly retrieve unique entity identifiers (QID), name and description. The QID of each technology served as a stable reference throughout the pipeline. Next, we take names of each technology and query them in arXiv through their API, and for each technology we extracted at maximum 100 papers. Extracted data includes paper id, publishing date, title, summary and

authors. The paper ids also served as a unique identifier for research papers throughout the pipeline. To query for start-ups, we first used emerging technology keyword list to search and retrieve data entries of start-ups working on said technology in the YCombinator dataset. We extracted their names, founders, founding year, industry, description, region, tags and website. We then augmented the data with a snapshot of Crunchbase 2014, extracting their total funding and more detailed funding information such as seed, venture and crowdfunding. We further enriched the data on start-ups with a sample from BrightData-scraped Crunchbase. For the extraction of job skills, we first query job positions of start-ups through a open-sourced module StaffSpy, and then we query the job positions we extracted over the Kaggle LinkedIn Job Skills Dataset to infer skill associations to start-ups.

3.2 Data Cleaning and Normalization

- **start-up Names:** Each start-up names were simplified to converting into lowercase and removing non-alphabetical characters. This normalised version of name is used as a unique identifier for start-ups across the pipeline, while the original version was retained for traceability. We chose this over assigning artificial unique IDs because data on start-ups especially came from multiple sources with no common identifier. Assigning artificial unique ID would only add unnecessary complexity and potential errors.
- **Skills:** We normalized skills by standardizing text format to ensure consistency, but we intentionally avoided fuzzy matching as it would conflate semantically distinct entries (e.g., "Java" and "Javascript"). Multilingual entries were not translated as they might involve LLM for processing and hence would induce great cost and complexity.
- **start-up Founding Dates and Funding:** We check multiple columns to find which dataset contains the most complete fields and fallback to use the next value available if an empty field is found. If the field is missing in all datasets, it is left null.
- **Location:** They are extracted and standardized by parsing the strings or JSON fields; then they are mapped to regions using a predefined region map.

3.3 Entity Linking and Deduplication

- **start-ups:** Records from Crunchbase and BrightData are merged with YCombinator by their normalized names, prioritizing YCombinator's descriptive fields, and Crunchbase and BrightData's financial metadata.
- **Technologies:** A manually curated synonym list was created and stored as a JSON format to group different terms referring to the same technology. This controlled vocabulary helped in matching across papers, start-ups, and WikiData entities.
- **Papers to Technologies:** Fuzzy matching was applied to match the title and abstract of the papers to technology terms.

3.4 Pipeline Orchestration and Configuration

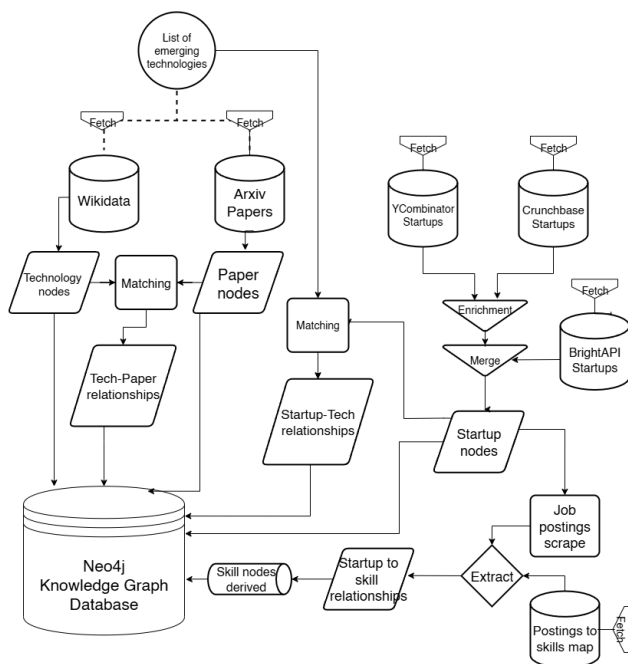


Figure 1 Pipeline Architecture

The pipeline is modular and cache-aware, enabling reproducibility and usability. It is orchestrated through `run_pipeline.py`, a top-level script coordinating all extraction, transformation, and loading (ETL) steps.

Configuration is controlled via environment variables such as `USE_CACHE`, `SCRAPE_JOBBOARD` and `LOAD_SKILLS`. These options allow the pipeline to switch between cached and fresh runs, scrape

optional data, or selectively activate modules. If the variables are unset, the pipeline will prompt for user input. Environmental variables were chosen to simplify automated, Dockerized deployments, while interactive prompts enhance ease-of-use during local experimentation and debugging.

The pipeline's codebase is structured into clearly defined located within the `\src` directory. Each script encapsulate a specific stage of workflow:

- **Data Extraction:** `get_arxiv.py`, `get_crunchbase.py`, `get_wikipedia.py`, `get_jobboard.py`
- **Data Cleaning and Normalization:** `clean_data.py`
- **Graph Loading:** `load_to_neo4j.py`

This modular design ensures that each component could be independently developed, tested or replaced, supporting iterative development and easier debugging.

To ensure robustness during graph loading, especially in cases where Neo4j may take time to initialize, the pipeline includes a `wait_for_neo4j` utility. This function pauses execution until Neo4j is confirmed to be live and accessible, minimizing connection-related failures. It also validates the presence of necessary cached files before execution, providing immediate feedback if prerequisites are missing.

The deduplication logic throughout the pipeline relies on normalized startup names to act as identifiers. While this has proven effective for the current dataset, future expansions involving multilingual or semantically complex entities may require more advanced resolution strategies. For the current scope, this approach strikes a balance between reliability, transparency, and simplicity.

3.5 Graph Construction

The knowledge graph schema was designed to model key entities with their relationships, with nodes representing technologies, research papers, start-ups, and skills, and edges representing relationships. Each node is assigned a unique identifier, and the original fields were preserved as properties for auditability.

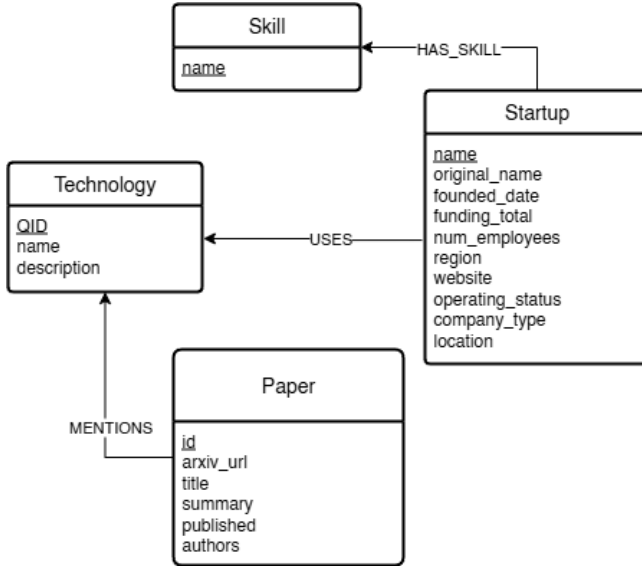


Figure 2 Graph Schema

To manage large and complex relationships and ensure accurate and meaning relationships, we use strict matching logic (such as high threshold for fuzzy matching) and preserve only the highest-confidence matches. We also retain original data fields for traceability, allowing us to audit or refine relationship creation if needed. This combination of in-memory deduplication, careful batching, and conservative matching criteria helps maintain both the integrity and the interpretability of the graph’s relationships.

4 Evaluation

4.1 Graph Coverage and Completeness

Since technology is the starting and originating point of our pipeline, all technology node except one, is connected to at least one paper and one start-up. That exception being the 6G network has yet to have any start-up working on it is because it is still early on the stages of standardization and not yet commercialized. In other words, every research paper and start-up is connected to a technology node. However, the coverage of Skill nodes are less-than-satisfactory. Only 165 out of 5904 (2.7%) of Startup nodes are connected to at least one Skill node. While the focus of this project is building the pipeline itself, instead of a comprehensive Knowledge Graph to be analysed for meaningful insights, we would like to address the low coverage regarding Skill nodes. The biggest reason is that we are obtaining skill data and how they are matched with start-ups and job positions through

scraping job boards. While this is feasible in a small scale, if we would like to do a full-scaled run on the pipeline, ethical and legal issues might arise from scraping thousands of profiles. It would also take a very long time and a lot of resources to support a full-scaled run.

Out of the 23 technology nodes we have currently extracted, 22 of them (95.6%) has over 100 papers mentioning them, and 10 of them has no less than 10 start-ups working on them. The Knowledge Graph currently contains 14,050 nodes and 24,640 relationships, with each node having an average of 1.75 relationships.

When we look at completeness of data, we found out that for Technology, Paper and Skill, none of the nodes had any empty fields, due to the Skill node only having one attribute, and the well-structured and high quality data entries we obtained from WikiData and arXiv. However, regarding the completeness of Startup nodes, there are 901 out of 5904 nodes that contain at least one empty field, making 15.3% of Startup nodes incomplete. That is largely due to the restriction of data quality we have from obtaining data from multiple sources with limited time scope and coverage, a trade off between the tremendously high budget of Crunchbase official API and a relatively incomplete data entries.

4.2 Challenges

Data Integration and Transformation: One of the most challenging aspects was merging and deduplicating startup records from multiple sources (YCombinator, Crunchbase, Brightdata), each with different naming conventions, missing fields, and inconsistent metadata. Integrating and cleaning funding and founding date fields was also particularly tricky. Each source used different formats, sometimes with missing or conflicting information, so we had to develop fallback logic and prioritize the most reliable columns. Handling edge cases—such as dates given as years, ambiguous or malformed funding JSON, or empty strings labelled as “null”—required repeated refinement.

Matching and Entity Resolution: One of the things that also required frequent iteration was our matching logic for technologies and startups, as tuning the fuzzy matching thresholds and handling syn-

onyms or acronyms proved to be a delicate balance between recall and precision. This challenge is actually a manifestation of a broader, well-known problem in computer science: entity resolution and semantic matching, where mapping diverse real-world texts to a single concept is inherently difficult. Our approach (using a manually curated synonym list for technologies) helped address some of the most obvious variations, but it is ultimately a heuristic solution. While effective for our current scope, it cannot capture the full complexity of language, and more advanced methods (such as embedding-based similarity or knowledge-based disambiguation) would be needed for larger or more ambiguous datasets.

4.3 Trade Offs

Overall, we consistently favoured solutions that were robust, transparent, and easy to maintain, even if it meant accepting some limitations in completeness or flexibility. This balance allowed us to deliver a reliable pipeline that can be iteratively improved as needs evolve. Some of these include:

- A manually curated synonym list for technologies, which is easy to maintain and audit, but not as comprehensive as automated or AI-driven approaches. Kept the system simple and reliable, at the cost of some recall.
- For performance, we batch all deduplication and relationship creation in memory before loading to Neo4j, which is fast for our dataset size but may not scale to much larger graphs without further optimization.
- Avoiding adding translation or external API calls for skills, prioritizing speed and cost control over full multilingual coverage.
- Simplicity in configuration: environment variables and interactive prompts are easy for users to understand and modify, but do not support the most advanced or dynamic configuration scenarios.

4.4 Possible Improvements

- **Plan for versioning and incremental updates:** Right now, the pipeline is optimized for full-runs. From the beginning, we could've included logic for detecting diffs and only updating or re-enriching affected nodes.

- **Use a centralized ID system earlier:** While name normalization worked well, a lightweight global ID system for startups could've helped track duplicates more reliably—especially across ambiguous or multilingual names. Example: generate hash-based pseudo-IDs at ingestion time.
- **More advanced skill/technology linking:** Use embeddings or classification models (e.g., BERT-based) to detect semantic similarity between startup text and tech/skill vocabularies. This would improve matching beyond string-based heuristics.

5 Conclusion

This methodology outlines a modular, data-driven approach to constructing a Knowledge Graph for emerging technologies by integrating datasets institutional, open, and community-driven sources. Through normalization, deduplication and selective enrichment process, we ensured consistent and traceable merging of records across sources. Pragmatic decisions enabled us to maintain both accuracy and interpretability. The pipeline itself is designed for reproducibility and modularity, orchestrated via a centralized controller script and automated by Docker, and structured into logically separated modules for extraction, transformation, enrichment and graph loading.

While some component, such as skill interface and multilingual support, remain areas for improvement, our current implementation offers a robust baseline for analysing the relationships between technologies, research papers, start-ups and skills. Ultimately, this project presents a feasible and effective approach to structuring fragmented innovation data to a unified graph-based representation. It lays the groundwork for more advanced applications in technology scouting, trend analysis, and knowledge discovery across the innovation ecosystem.

References

- [Ferré and Langlais, 2023] Ferré, A. and Langlais, P. (2023). An analysis of entity normalization evaluation biases in specialized domains. *BMC Bioinformatics*, 24(1).
- [Stegeman, 2024] Stegeman, J. (2024). What is a knowledge graph?